Travaux Pratiques : CI/CD avec Jenkins, Docker et SonarQube

TP 1 : Installation et découverte de Jenkins

Objectif : Déployer Jenkins via Docker et configurer l'environnement de base.

1. Démarrez un conteneur Jenkins en utilisant l'image officielle `jenkins/jenkins:lts-jdk17`. Assurez-vous d'exposer le port 8080 et de monter un volume pour persister les données.
2. Récupérez le mot de passe administrateur initial dans les logs du conteneur.
3. Connectez-vous à l'interface web, installez les plugins suggérés et créez votre compte administrateur.
4. Explorez l'interface et identifiez où configurer les outils globaux (Maven, JDK, Docker).

TP 2 : Compilation d'un projet Java avec Maven

Objectif : Créer un premier job Jenkins pour compiler et tester un projet Java.

1. Créez un nouveau Job de type "Pipeline".
2. Écrivez un script Pipeline (Jenkinsfile) basique avec deux étapes : "Checkout" et "Compilation".
3. Utilisez l'agent Docker avec l'image `maven:3.9-eclipse-temurin-17` pour exécuter les commandes Maven.
4. Ajoutez une étape "Tests" pour exécuter `mvn test` et publiez les résultats des tests JUnit.
5. Lancez le build et vérifiez que les tests passent avec succès.

TP 3 : Conteneurisation avec Docker

Objectif : Intégrer la construction d'une image Docker dans le pipeline.

1. Dans votre projet Java, créez un fichier `Dockerfile` utilisant un build multi-étapes (multi-stage) : une étape pour compiler avec Maven, et une étape finale avec une image JRE légère pour exécuter l'application.
2. Modifiez votre Pipeline Jenkins pour ajouter une étape "Build Image Docker".
3. Assurez-vous que Jenkins a accès au socket Docker de l'hôte pour pouvoir construire l'image.
4. Lancez le pipeline et vérifiez avec `docker images` que votre image a bien été créée.

TP 4 : Analyse de qualité avec SonarQube

Objectif : Intégrer l'analyse de code SonarQube dans le pipeline d'intégration continue.

1. Démarrez un serveur SonarQube via Docker sur le port 9000.
2. Générez un token d'authentification dans SonarQube.
3. Configurez l'intégration SonarQube dans Jenkins (System Configuration) et ajoutez le token dans les "Credentials" de Jenkins.
4. Ajoutez une étape "Analyse SonarQube" dans votre Pipeline en utilisant la commande `mvn sonar:sonar`.
5. Observez les résultats de l'analyse sur le tableau de bord SonarQube.

TP 5 : Le pipeline CI/CD complet

Objectif : Assembler toutes les étapes pour obtenir un pipeline CI/CD robuste de bout en bout.

1. Reprenez votre Jenkinsfile et assurez-vous qu'il contient toutes les étapes : Checkout, Compilation, Tests, Analyse Qualité (SonarQube), et Build Docker.

2. Ajoutez une étape “Quality Gate” qui met en pause le pipeline en attendant le résultat de l’analyse SonarQube. Si la qualité est insuffisante, le pipeline doit échouer.
3. Ajoutez un bloc `post` à la fin de votre pipeline pour nettoyer l’espace de travail (`cleanWs()`) et afficher un message de succès ou d’échec.
4. Exécutez le pipeline complet et vérifiez chaque étape dans l’interface “Pipeline Graph View” de Jenkins.